MICA-VLA: A Memory-Integrated Semantic Control Architecture for Compact Vision–Language–Action Models

Abstract— **Previous studies have linked language generalization difficulties in vision–language–action (VLA) policies to language forgetting, modality imbalance, and visual shortcuts. Inspired by the division of responsibilities in cascade control, we further identify that reliable language control requires not only preserved language representations but also a stable task reference that continuously informs observation-based action decisions. In an appropriate division of responsibilities within a VLA policy, language should specify goals, object roles, and execution constraints, while vision and robot state provide the current execution conditions; these inputs must remain effectively connected and integrated during action decisions. Based on this insight, we propose MICA-VLA, a memory-integrated semantic control architecture for compact policies. Policy Semantic Memory (PSM) retrieves frozen instruction anchors using the policy's native embeddings, retaining the original instruction when similarity and margin gates reject a match. After the selected instruction is contextualized with the current images, the Semantic Control Expert (SCE) continuously uses its contextualized features in action updates through duplex semantic routing. On both the LACE real-robot tasks and the public LIBERO-Para benchmark, MICA-VLA achieves better execution performance than the comparison baselines, showing adaptation to unseen wording and improved instruction following in real-robot object-role reversal tasks.**

I. INTRODUCTION

Enabling robots to perform diverse manipulation tasks across different scenes based on natural language is an important goal of robot learning [1], [2]. Recent vision–language–action (VLA) models and generalist robot policies combine visual perception, language conditioning, and action generation, offering a new route toward this goal [3]–[7]. By drawing on pretrained vision–language model representations and learning actions from robot demonstrations, VLA policies translate natural-language requirements into manipulation behavior [3], [4]. Meanwhile, compact policies such as SmolVLA further reduce training and inference costs, bringing language-conditioned manipulation closer to deployment on resource-constrained robots [8]–[10]. As manipulation capabilities and deployment efficiency improve, whether a policy still executes correctly when instruction wording and task requirements change becomes central to its practical value.

Recent studies have revealed two difficulties in VLA language control. Kim et al. found in LIBERO-Para that synonymous paraphrases reduce the execution performance of multiple VLA policies [11], whereas Hou and Zhao found through LangGap that $\pi_{0.5}$ may fail to adapt its behavior to changed object or goal requirements within the same scene [12]. Xu et al. link language forgetting to modality imbalance [13], while Fang et al. identify dataset-induced visual shortcuts in counterfactual instruction-following failures [14]. These diagnoses connect to broader findings on shortcut learning [15] and causal misidentification in imitation learning [16]. Existing responses broaden language supervision through augmentation and instruction tuning [17], [18], or strengthen language conditioning through learning objectives and inference-time guidance [19], [20]. These studies indicate that expression adaptation and semantic use are interconnected aspects of language control. We further examine their connection: how can a compact policy retain a stable task reference under synonymous paraphrases while continuously using its goals, roles, and constraints to guide action decisions and respond appropriately when task requirements change?

Drawing on the distinction between reference signals and measured feedback in control [21], we analyze the respective responsibilities of language and observations. Vision and robot state describe the current environment and the robot itself, but cannot alone determine which task the robot should perform when a scene permits several operations. Language instructions specify the manipulation goal, object roles, and execution constraints. For example, "move the beaker, avoid the cylinder" and its equivalent paraphrases should preserve the same task requirements; changing the instruction to "move the cylinder, avoid the beaker" must exchange the manipulated and avoided objects even when the scene and most words remain unchanged (Fig. 1(a,b)). This distinction is especially important in laboratory manipulation because completing a grasp or transfer does not necessarily satisfy object-selection and avoidance requirements. We therefore summarize the design requirements for reliable language control as two connected stages: retaining a task reference that is stable across expression changes at the input, and continuously using its task requirements with current observations at the action end.

Based on this reasoning, we propose MICA-VLA, a memory-integrated semantic control architecture that unifies instruction-reference selection with semantically conditioned action generation. PSM retrieves training instructions from a frozen nonparametric memory using native policy embeddings, accepting matches through similarity and margin gates and otherwise retaining the original input. This input-side selection supplies a familiar expression for a new wording of a known task, but selecting an instruction does not ensure its behavioral use. SCE therefore reads the selected instruction's image-contextualized features and repeatedly introduces them after environment-conditioned action updates through Duplex Semantic Routing (DSR). The two components are designed

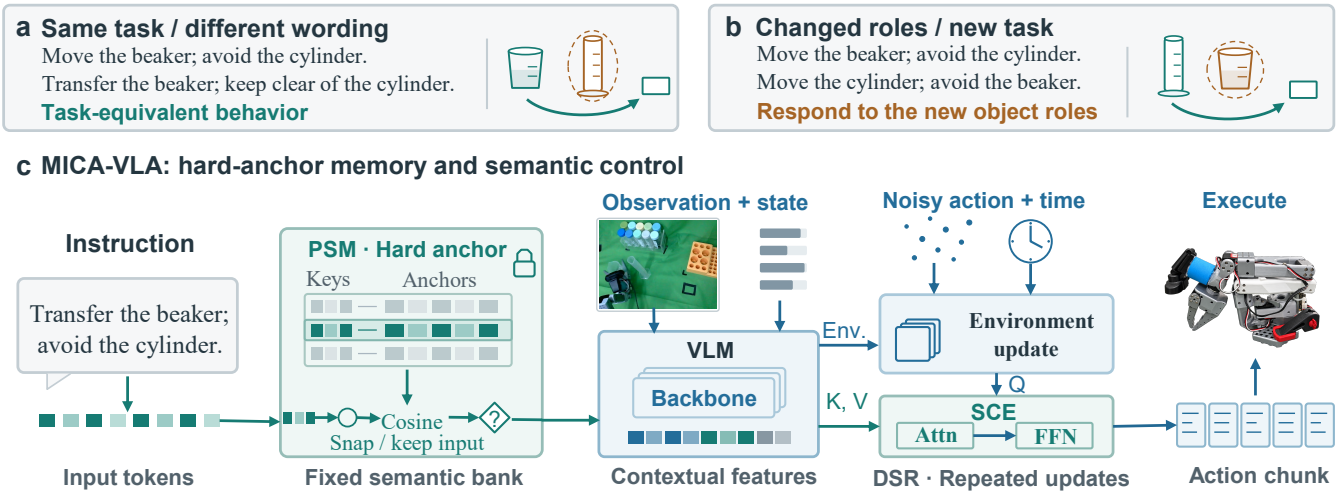


Fig. 1. **Memory-integrated semantic control.** (a) Equivalent wordings should preserve task requirements; (b) object-role changes should alter behavior. Both depict design requirements. (c) The anchor-replacement path is shown. PSM replaces tokens and their mask when both gates accept; the fallback path retains the original instruction. SCE uses the resulting contextualized language features through Duplex Semantic Routing (DSR).

to improve instruction selection and the behavioral use of the selected instruction, respectively.

We evaluate MICA-VLA on 17 real-robot manipulation tasks in LACE and on the public LIBERO-Para benchmark [11]. LACE tests responses to expression changes and task requirements through held-out wording, object-role reversals, and avoidance requirements. The real-robot results show that, relative to SmolVLA, MICA-VLA increases mean held-out instruction success from 42.1% to 76.7% and mean success on two role-reversal tasks from 14.0% to 86.0%. LIBERO-Para provides large-scale instruction paraphrases while preserving task semantics and scenes, allowing further analysis of wording robustness for known tasks. Through a four-arm comparison of PSM and SCE, we examine the contributions of instruction memory and semantic control, as well as their combined execution performance.

The main contributions of this paper are as follows.

- We propose MICA-VLA, a memory-integrated semantic control architecture for compact VLAs that connects input-side task-reference selection with action-side semantic use through the control responsibilities of task requirements and feedback observations.
- We design PSM to select training instruction anchors through native policy embeddings and SCE to repeatedly condition action updates on the selected instruction through duplex semantic routing.
- We conduct complementary evaluations on LACE real-robot tasks and the public LIBERO-Para benchmark, testing wording changes, object roles, and avoidance requirements, alongside component comparisons, to evaluate execution performance and analyze the roles of the components.

II. RELATED WORK

A. Language Robustness and Instruction Following

Methods for improving instruction following differ in where they intervene between language supervision and action prediction. Synthetic augmentation expands the expressions available during training [17], while InstructVLA couples vision–language understanding with action instruction tuning [18]. Beyond expression coverage, LangForce contrasts a visual prior with a language-conditioned policy through Bayesian decomposition to strengthen the instruction information contained in actions [19]. Residual Semantic Steering extends this intervention to inference, combining paraphrase supervision with residual guidance during action generation [20]. The distinction between broadening supervision and changing its behavioral use motivates our focus on how a selected task reference reaches the action policy.

Corresponding evaluations also distinguish wording robustness from responses to task semantics. LIBERO-PRO and LIBERO-Plus test controlled instruction and environmental perturbations [22], [23]. Complementing these benchmarks, multi-object picking studies distinguish basic manipulation from instruction-grounded success [24]. LIBERO-Para isolates wording changes while preserving task semantics and scenes [11]. Following this distinction, we combine paraphrase evaluation for known tasks with behavioral tests involving object roles and avoidance constraints to examine task-reference stability and its role in action decisions.

B. Retrieval- and Memory-Augmented Policies

Nonparametric retrieval can reuse learned representations without further training: kNN-LM interpolates language-model predictions with a token distribution retrieved using pretrained context representations [25], whereas VINN predicts actions from neighboring visual demonstrations [26]. Memory-augmented VLA policies extend retrieval to task-execution context. MAP-VLA constructs task-stage memory from historical demonstrations, encodes stage information as learnable prompts, and retrieves relevant memories based on trajectory similarity to assist action generation [27]. After retrieval-conditioned training, Retrieve, Don't Retrain freezes the policy and uses cross-embodiment trajectories as motion

priors for new tasks [28]. RA-VLA combines behavior-aligned context retrieval with execution mechanisms, promoting the conversion of retrieved demonstration information into actions [29].

These VLA methods primarily reuse task-execution experience, with memory contents comprising stages, trajectories, or demonstration contexts. PSM instead uses training instruction anchors as memory units, reuses native policy token embeddings for retrieval, and feeds the selected anchor into the language input, operating at task-reference selection before action generation. This distinction allows PSM to reuse instruction experience for wording adaptation on known tasks without an additional retrieval encoder or trainable memory parameters.

C. Modality Routing and Action Generation

Language-conditioned control depends on both the information encoded in representations and the computation through which that information affects actions. Visual pretraining supplies control features [30], [31]; language–image learning and temporal alignment connect such representations to language-conditioned behavior [32], [33]. Contrastive image–text learning [34] and predictive coding [35] provide related representation-learning foundations, but the policy must still determine how to use the resulting features.

Conditioning architectures address this latter question by assigning features specific roles in the computation. FiLM modulates visual features through language-conditioned affine transformations [36], whereas CLIPort combines semantic and spatial pathways for manipulation [2]. Within VLAs, CogVLA uses instruction-driven routing and sparsification [37], and NanoVLA combines vision–language decoupling with dynamic routing [10]. ReconVLA instead supervises task-relevant visual representations through target-region reconstruction [38]. These approaches distinguish selecting or grounding multimodal information from specifying where it conditions action generation.

At the action-generation end, the location and repetition of conditioning become explicit architectural choices. RDT-1B alternates image and language conditioning across adjacent layers [39], VLA-Adapter passes layerwise VLM features to the action policy through bridge attention [40], and SmolVLA interleaves cross-attention and causal self-attention within its action expert [8]. SCE builds on conditioned action generation but organizes its updates around the control responsibilities identified in our analysis. Its semantic pathway follows each environment-conditioned update, connecting the instruction reference selected by PSM to the evolving action representation.

III. MICA-VLA

A. Problem Formulation and Overall Interface

The control-responsibility analysis in Section I leads to an architectural distinction between the task reference and the observations used to execute it. Because the same scene can support different tasks, observation-conditioned actions must remain connected to the goals, object roles, and constraints specified by language. MICA-VLA implements this connection by selecting an instruction reference and then using it throughout observation-conditioned action generation. Given an instruction ℓ and observations O_t, instruction memory $\mathcal{M}$ supplies a reference $L^\star$ that conditions the action chunk a (Fig. 1(c)):

$$L^\star = f(\ell; \mathcal{M}), \qquad a \sim \pi_\theta(\cdot \mid O_t, L^\star). \tag{1}$$

Here, $L^\star$ comprises instruction tokens and their attention mask, providing a common interface between PSM and SCE. PSM selects this reference, whereas SCE conditions action updates on the task information it retains. We call the design criterion linking reference formation to behavioral use semantic control continuity. Its behavioral objectives are task-equivalent execution under equivalent wording, rather than identical trajectories, and corresponding responses when goals, roles, or constraints change.

B. Policy Semantic Memory

To form the reference, Hard-Anchor PSM retrieves a familiar training instruction for a new wording of a known task, subject to a dual acceptance gate. Using the policy's native token embeddings places the query and stored anchors in the same representation space without an additional retrieval encoder. For token IDs x and mask m, the pre-Transformer embedding layer e_θ produces an L_2-normalized masked mean:

$$\phi_\theta(x, m) = \mathrm{norm}_2\left(\frac{\sum_r m_r e_\theta(x_r)}{\sum_r m_r}\right). \tag{2}$$

Each memory entry (k_i, x_i, m_i, c_i) stores a key, token IDs, mask, and task-class metadata. Keys $k_i = \phi_\theta(x_i, m_i)$ and queries $q = \phi_\theta(x, m)$ use the same checkpoint and tokenizer. These text-only keys determine anchor selection before multimodal contextualization; they are not the language features used by SCE. The masked mean is invariant to permutations of an identical set of valid tokens, including their multiplicities. Retrieval searches all task classes without task-label filtering or scene-based reranking. It scores each task by its best anchor, so synonymous anchors do not compete as different tasks:

$$\begin{aligned} s_c &= \max_{i:c_i=c} q^\top k_i, \quad c^\star = \arg\max_c s_c, \\ \Delta s &= s_{c^\star} - \max_{c \neq c^\star} s_c. \end{aligned} \tag{3}$$

Let $i^\star$ be the highest-scoring anchor in $c^\star$. Both the absolute similarity and task-class margin must pass their thresholds before replacement:

$$L^\star = \begin{cases} (x_{i^\star}, m_{i^\star}), & s_{c^\star} \geq \tau_a \text{ and } \Delta s \geq \tau_m, \\ (x, m), & \text{otherwise.} \end{cases} \tag{4}$$

Low-score or ambiguous matches thus retain the original instruction rather than forcing an anchor. Both paths preserve a complete token sequence and its word order for VLM

contextualization; SCE uses whichever instruction this interface supplies. To examine the effect of rejection, we also evaluate forced Top-1 on LIBERO-Para, bypassing both gates and always supplying $(x_{i^\star}, m_{i^\star})$. Neither retrieval setting certifies semantic correctness or physical safety.

C. Semantic Control Expert

Selecting a familiar instruction does not by itself ensure that its roles and constraints govern execution. In mirror tasks, the same objects exchange manipulation and avoidance roles, requiring different actions within an otherwise unchanged scene. SCE addresses this behavioral use by reading the selected instruction's image-contextualized language features H_t^ℓ, rather than PSM's pooled retrieval key. Let E_t denote the environment and robot-state conditions supplied to the action backend.

Duplex Semantic Routing (DSR) translates the control-responsibility distinction into successive environment-conditioned updates and semantic modulation. The environment pathway first updates the action representation from current conditions; the semantic pathway then adjusts that representation using the selected instruction. Repeating this sequence gives task requirements an explicit conditioning position as action features develop across network depth. At each denoising step, noisy actions and time embeddings initialize h_0, and expert cycle j computes

$$\begin{aligned} \widetilde{h}_j &= F_j^{\mathrm{env}}(h_j, E_t), \\ u_j &= \widetilde{h}_j + \mathrm{CA}_j\Big(\widetilde{h}_j; K_j(H_t^\ell), V_j(H_t^\ell)\Big), \\ h_{j+1} &= u_j + \mathrm{FFN}_j(u_j). \end{aligned} \tag{5}$$

Here, j indexes network depth, not denoising time, and normalization is omitted for clarity. Cross-attention (CA) uses $\widetilde{h}_j$ as queries and projections K_j, V_j of H_t^ℓ as keys and values, followed by a feedforward update (FFN) [41]. Thus, the observation-conditioned representation supplies the query that determines how language features contribute to the next action update. Semantic modulation is a learned conditional representation update, not a separate constraint solver.

To implement this routing, the environment attention mask blocks direct access to language tokens, while the semantic pathway reads H_t^ℓ. Contextualized environment features may still contain language information, so the separation concerns direct attention access rather than complete modality independence. The implementation uses eight cycles with independent modules and zero-initializes the semantic attention output projections. This initialization limits early residual perturbations, following the conditioning principle used in ControlNet [42].

D. Training and Inference

The two components' roles lead to different training requirements. SCE learns its conditional updates through SmolVLA's action-learning interface [8] and flow-matching objective [43]. PSM instead adds no trainable parameters, and its discrete retrieval receives no gradients. Its anchors are encoded offline after training, using the selected checkpoint so that memory keys and inference queries share the same embedding layer. The checkpoint and memory remain frozen throughout evaluation, with no online memory updates; changing the embedding layer requires re-encoding the anchors.

Inference follows the same division of responsibilities, proceeding from tokenization and gated reference selection to multimodal contextualization and action generation. Disabling PSM passes the original tokens $L^\star = (x, m)$ to the same backend without changing the subsequent computation. As in the receding-horizon execution used by Diffusion Policy [44], new observations condition the next action decision after execution, closing the feedback loop under the selected task reference. The following evaluation protocol tests this architecture through complementary changes in wording and task requirements.

IV. EXPERIMENTAL SETUP

A. Laboratory Scenes and Task Design

LACE (Language-conditioned Avoidance in a Chemistry-lab Environment) comprises 17 laboratory manipulation tasks on a single robot platform to examine how language requirements change manipulation choices. Beakers, graduated cylinders, test tubes, and racks have similar appearances but can serve as targets or obstacles under different instructions. Their changing roles allow task requirements to vary while the physical environment remains unchanged. The same setting also distinguishes successful manipulation from instruction compliance, since a completed transfer may still involve contact with an object to be avoided.

Accordingly, the task set progresses from five vision-dominant tasks testing basic skills to five referential tasks requiring the correct object or destination. Seven relational tasks add avoidance constraints (Fig. 3). Within this last category, two mirror tasks exchange the roles of a beaker and a cylinder in the same scene. This reversal makes the instruction determine which object to move and which to avoid, complementing the equivalent paraphrases used to test expression stability.

B. Real-Robot Platform and Instruction Configuration

These language comparisons use a common execution platform, a single SO-100 arm [45] equipped with top, side, and wrist cameras (Fig. 2). To reduce mechanical initialization differences, platform calibration requires each joint's homing deviation to remain within 4° and checks the full gripper range.

For each known manipulation task, the instruction set distinguishes familiar training expressions from unseen wording. It contains one base instruction, four in-distribution paraphrases, and one verb-disjoint held-out instruction. The LACE PSM bank, however, stores only the original instruction for each task, totaling 17 anchors; neither training paraphrases nor held-out wording are stored. Real-robot evaluation uses $\tau_a = 0$ and $\tau_m = 0.05$, retaining the original input when either gate rejects the match. Each task is evaluated with

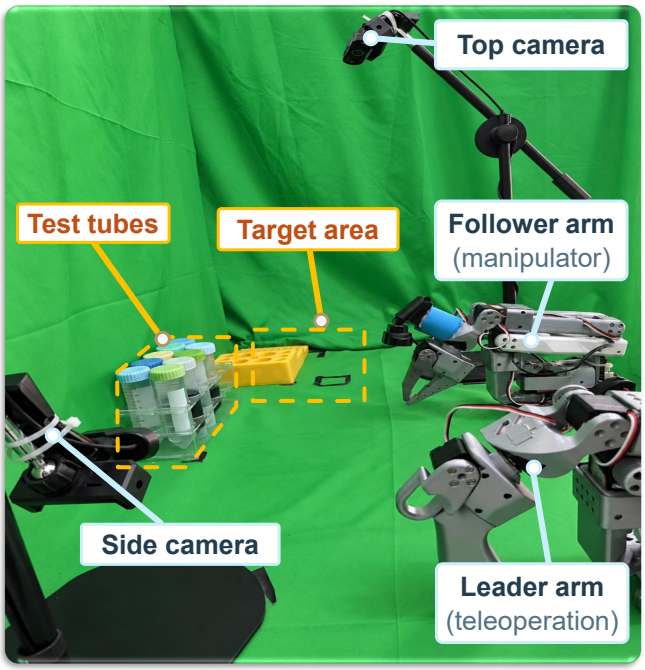


Fig. 2. **LACE experimental platform.** The SO-100 leader–follower setup is used for teleoperation. The top and side cameras, test tubes, and target area are labeled; the wrist camera is not labeled in this view.

50 rollouts per condition, with the instruction and outcome linked to each episode for the comparisons below.

C. Real-Robot Comparisons and Behavioral Metrics

To evaluate instruction selection and its behavioral use, the four-arm design switches PSM off or on for the Vanilla and SCE backbones. Vanilla + PSM corresponds to MICA-VLA without SCE, SCE alone removes PSM, and SCE + PSM is the complete configuration. Table II abbreviates these removals as "w/o SCE" and "w/o PSM." Switching PSM reuses the same checkpoint, whereas the Vanilla and SCE backbones are trained separately, distinguishing inference-side memory effects from backend configuration differences.

For both wording changes and role reversals, success requires completing the task specified by the current instruction. Table I first compares success rate (SR) under Base, the original training-seen instruction, and Para, the verb-disjoint held-out paraphrase. This comparison preserves task meaning while changing expression; Para does not aggregate the four in-distribution paraphrases. Category means average tasks within each category, and the overall mean averages all 17 tasks.

The symbol $\star$ marks mirror tasks, and ΔPara denotes MICA-VLA minus SmolVLA Para SR in percentage points; means and differences are rounded independently.

Because completing a manipulation can still violate an avoidance instruction, Table II supplements task SR with separate role and constraint measures. Mirror SR is the mean held-out instruction success rate over the two role-reversal tasks. Across the seven relational tasks, action completion (AC) counts trials completing the basic manipulation, while constraint violation (CV) counts completed manipulations that violate instruction constraints. Both rates use all evaluation trials as the denominator, and incomplete manipulations are not counted as CV. Under this event definition, AC minus CV is the fraction that completes the operation without violating constraints. For the configurations shared by Tables I and II, these measures use the same held-out instructions and rollout records as Para SR.

With 50 rollouts per task and condition, each relational-task summary covers 350 trials per configuration, and each mirror summary covers 100 trials. We report descriptive task means and percentage-point differences; these rollouts measure execution outcomes for the evaluated configurations, not independent model repetitions.

D. Public LIBERO-Para Simulation Evaluation

Complementing LACE's role and constraint tests, public LIBERO-Para [11] evaluates meaning-preserving wording changes across 4,092 instructions, 10 evaluation tasks, and 43 variation combinations. The three categories comprise 870 action-expression, 259 object-expression, and 2,963 combined paraphrases; combined paraphrases refer to combinations of language transformations, not new task compositions. Training covers 40 base tasks in LIBERO [46], and the evaluation tasks belong to this known-task set, focusing the evaluation on expression robustness rather than learning new manipulation tasks.

For this public benchmark, the four configurations are labeled V00 (Vanilla), V10 (Vanilla + PSM), V01 (SCE only), and V11 (MICA-VLA). The Vanilla and SCE checkpoints were trained for 60,000 steps with training seed 1000 under a mixed40 base-instruction configuration without paraphrase augmentation. For V10 and V11, the corresponding checkpoint's native token embeddings independently encode the same 40 training base instructions into a frozen memory bank. We evaluate dual-gate PSM with $\tau_a = 0$ and $\tau_m = 0.05$, and forced Top-1 with both acceptance tests bypassed. Both settings use the same global anchors without scene-based candidate filtering or reranking. Retrieval adds no policy-training steps, and the memory remains frozen throughout evaluation.

To compare execution under corresponding conditions, the four arms and paraphrases of the same task share the initial state and paired denoising noise. Each configuration executes one rollout per instruction with evaluation seed 7. The dual-gate four-arm matrix contains 16,368 rollouts; adding the two Top-1 configurations gives 24,552, with the PSM-off results shared between comparisons. Each task uses the first state in its public initialization file, followed by 10 settling steps and at most 300 policy-control steps. Following SmolVLA's simulation inference setup [8], the policy generates relative-action chunks of length 50, executes one action before replanning from the new observation, and uses 10 denoising steps. Pairing is maintained at corresponding control steps, with noise determined by the scene identifier, initial state, evaluation seed, and control step. This construction is independent of configuration, wording, or run order; run

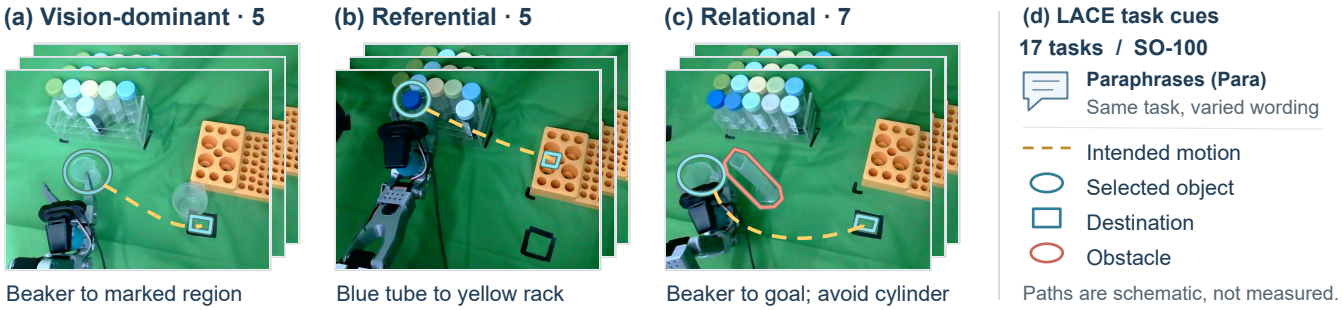


Fig. 3. **LACE tasks and instruction configuration.** (a–c) Vision-dominant, referential-selection, and relational-constraint categories contain 5, 5, and 7 tasks, respectively. (d) Task cues and paraphrase illustration; Para in Table I specifically denotes held-out wording. Dashed paths indicate intended motion, and outlines mark target objects, destinations, and obstacles.

TABLE I

LACE TASK SUCCESS RATES (%, HIGHER IS BETTER).

	SmolVLA		MICA-VLA		ΔPara
Task	Base	Para	Base	Para	
Vision-dominant (5)					
Pick beaker	88.0	68.0	88.0	80.0	+12.0
Grab centrifuge tube	74.0	60.0	78.0	72.0	+12.0
Open petri dish	58.0	22.0	56.0	48.0	+26.0
Clamp stir bar	86.0	66.0	84.0	80.0	+14.0
Place graduated cylinder	92.0	70.0	92.0	88.0	+18.0
Mean	79.6	57.2	79.6	73.6	+16.4
Referential (5)					
Colored tube → yellow	68.0	26.0	64.0	50.0	+24.0
Centrifuge tube → yellow	84.0	64.0	86.0	84.0	+20.0
Small → large beaker	90.0	66.0	88.0	92.0	+26.0
Clear tubes → yellow	86.0	60.0	84.0	80.0	+20.0
Pour, small → large	82.0	70.0	76.0	74.0	+4.0
Mean	82.0	57.2	79.6	76.0	+18.8
Relational (7)					
Beaker, avoid cylinder*	84.0	6.0	96.0	88.0	+82.0
Cylinder, avoid beaker*	80.0	22.0	96.0	84.0	+62.0
Funnel, avoid beaker	72.0	26.0	78.0	70.0	+44.0
Single tube, avoid beaker	76.0	34.0	84.0	86.0	+52.0
Single tube, avoid beakers	78.0	8.0	74.0	78.0	+70.0
Cylinder, avoid cylinders	84.0	20.0	86.0	80.0	+60.0
Single tube, avoid cylinder	74.0	28.0	72.0	70.0	+42.0
Mean	78.3	20.6	83.7	79.4	+58.9
Overall (17)	79.8	42.1	81.3	76.7	+34.6

TABLE II

BEHAVIORAL OUTCOMES UNDER HELD-OUT INSTRUCTIONS (%).

Model	Mirror↑	AC↑	CV↓
SmolVLA	14.0	78.6	58.0
w/o PSM	74.0	82.3	28.3
w/o SCE	12.0	77.1	22.6
MICA-VLA	86.0	89.7	10.3

records verify agreement in initial physical state, both camera images, and corresponding noise.

Success follows the environment's task-success criterion, and overall SR divides successful rollouts by all 4,092 evaluated instructions per arm. Category-specific SR further distinguishes performance across action, object, and combined expressions. We compare configurations using descriptive percentage-point differences calculated from unrounded success counts.

V. RESULTS

A. Performance Across 17 Real-Robot Tasks

We first examine whether the complete architecture improves execution under held-out wording across the LACE task set (Table I). MICA-VLA achieves higher Para SR than SmolVLA on all 17 tasks, increasing the overall mean from 42.1% to 76.7%, a gain of 34.6 percentage points. Breaking down this gain by task category, relational Para SR rises from 20.6% to 79.4%, exceeding the improvements on vision-dominant and referential tasks. The largest improvement therefore occurs in operations with more complex language requirements. To place this wording gain alongside familiar-instruction performance, we also compare Base SR, whose overall mean increases from 79.8% to 81.3%. The task-level Base changes comprise seven increases, two ties, and eight decreases of 2–6 percentage points.

B. Object Roles and Avoidance Constraints

The pronounced gains on relational tasks motivate a closer look at object-role responses and avoidance behavior (Table II). MICA-VLA achieves a mirror SR of 86.0%, compared with SmolVLA's 14.0%, indicating better responses to the current instruction when manipulation and avoidance roles are exchanged. To distinguish manipulation completion from constraint adherence, we examine the two behavioral scores assigned to the same relational-task rollouts. AC increases from 78.6% to 89.7%, while CV decreases from 58.0% to 10.3%. These subtask measures describe manipulation completion and constraint violations separately from the directly scored task-success endpoint.

The component-removal comparisons then examine how these outcomes change when instruction selection or SCE's semantic conditioning is removed. Without PSM, mirror SR is 74.0% and CV is 28.3%; without SCE, these values are 12.0% and 22.6%, respectively. The complete configuration achieves higher mirror SR and lower CV than either component-removal variant in these task records. Figure 4 illustrates object selection and contact events to clarify the behavioral criteria for success and violations.



Fig. 4. **Real-robot execution examples for four transfer-and-avoid tasks.** Blue outlines mark targets, orange dashed outlines mark objects to avoid, and red boxes indicate contact; frames are spaced 0.5 s apart. Symbols identify the displayed outcomes, and SR denotes the success rate in the corresponding task records.

TABLE III

LIBERO-PARA SUCCESS RATES UNDER DUAL-GATE AND FORCED TOP-1 RETRIEVAL (%, HIGHER IS BETTER).

Config.	PSM	Action	Object	Combined	Overall
V00	Off	14.48	30.50	4.08	7.97
V01	Off	28.28	45.17	10.56	16.52
V10	Dual gate	56.90	70.27	30.24	38.44
V11	Dual gate	61.38	76.83	33.14	41.91
V10	Top-1	76.67	83.01	59.60	64.71
V11	Top-1	**82.41**	**87.26**	**62.37**	**68.21**

C. Robustness to Public Paraphrases and Component Contributions

Having examined responses to roles and constraints on the robot, we turn to LIBERO-Para to test robustness across meaning-preserving expression changes. Table III compares dual-gate PSM and forced Top-1 against the same PSM-off references, with 4,092 instructions per configuration.

With rejection retained, dual-gate PSM raises Vanilla SR from 7.97% to 38.44%, while MICA-VLA reaches 41.91% compared with 16.52% for SCE alone. The four configurations yield 326, 1,573, 676, and 1,715 successes for V00, V10, V01, and V11, respectively. MICA-VLA exceeds Vanilla + PSM by 3.47 percentage points, showing an additional improvement beyond gated instruction selection.

We next bypass the gates to assess how extending anchor replacement affects execution. Forced Top-1 raises Vanilla + PSM to 64.71% (2,648 successes) and MICA-VLA to 68.21% (2,791 successes). These are increases of 26.27 and 26.30 percentage points over their dual-gate counterparts. MICA-VLA remains ahead by 3.49 percentage points; under both retrieval settings, it also outperforms Vanilla + PSM in every paraphrase category.

To examine the gain from bypassing the gates, we inspect replacement coverage. Each dual-gate backend replaces 1,671 inputs, retains 2,420 because of low margin, and finds one exact anchor match. Because LIBERO-Para preserves known-task meanings, this comparison tests broader anchor use under paraphrasing, not safety when task requirements change.

VI. CONCLUSION

Our control-responsibility analysis motivates a design insight: language supplies the task requirements, while

vision and robot state supply feedback for executing them. MICA-VLA implements this distinction through connected stages of instruction selection and behavioral use. PSM retrieves familiar training instructions with an original-input fallback, while SCE repeatedly conditions action updates on the resulting instruction's contextualized features. The LACE results report improvements under held-out wording and object-role reversals, while LIBERO-Para extends the evaluation to large-scale paraphrases of known tasks. In simulation, MICA-VLA improves over Vanilla + PSM under both dual-gate and forced Top-1 retrieval; bypassing rejection further increases SR on known-task paraphrases.

Together, these stages connect the choice of a task reference to its use during observation-based execution. This organization gives compact VLAs a concrete architecture for combining instruction memory with semantic conditioning, evaluated here through complementary wording, role, and constraint tests.

REFERENCES

[1] E. Jang, *et al.*, "BC-Z: Zero-shot task generalization with robotic imitation learning," in *Proc. CoRL*, vol. 164, 2022, pp. 991–1002.

[2] M. Shridhar, L. Manuelli, and D. Fox, "CLIPort: What and where pathways for robotic manipulation," in *Proc. CoRL*, vol. 164, 2022, pp. 894–906.

[3] B. Zitkovich, *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. CoRL*, vol. 229, 2023, pp. 2165–2183.

[4] M. J. Kim, *et al.*, "OpenVLA: An open-source vision-language-action model," in *Proc. CoRL*, vol. 270, 2025, pp. 2679–2713.

[5] K. Black, *et al.*, "π_0: A vision-language-action flow model for general robot control," in *Proc. RSS*, 2025.

[6] D. Ghosh, *et al.*, "Octo: An open-source generalist robot policy," in *Proc. RSS*, 2024.

[7] NVIDIA, *et al.*, "GR00T N1: An open foundation model for generalist humanoid robots," 2025, arXiv:2503.14734.

[8] M. Shukor, *et al.*, "SmolVLA: A vision-language-action model for affordable and efficient robotics," 2025, arXiv:2506.01844.

[9] J. Wen, *et al.*, "TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robot. Autom. Lett.*, vol. 10, no. 4, pp. 3988–3995, 2025.

[10] J. Chen, J. Wang, L. Chen, C. Cai, and J. Lu, "NanoVLA: Routing decoupled vision-language understanding for nano-sized generalist robotic policies," 2025, arXiv:2510.25122.

[11] C. Kim, M. Kim, M. Kang, H. Kim, and D. Jung, "LIBERO-Para: A diagnostic benchmark and metrics for paraphrase robustness in VLA models," 2026, arXiv:2603.28301.

[12] Y. Hou and L. Zhao, "LangGap: Diagnosing and closing the language gap in vision-language-action models," 2026, arXiv:2603.00592.

[13] K. Xu, *et al.*, "Seeing to act, prompting to specify: A Bayesian factorization of vision language action policy," 2025, arXiv:2512.11218.

[14] Y. Fang, *et al.*, "When vision overrides language: Evaluating and mitigating counterfactual failures in VLAs," 2026, arXiv:2602.17659.

[15] R. Geirhos, *et al.*, "Shortcut learning in deep neural networks," *Nat. Mach. Intell.*, vol. 2, no. 11, pp. 665–673, 2020.

[16] P. de Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[17] D. Shin, "Enhancing linguistic generalization of VLA: Fine-tuning OpenVLA via synthetic instruction augmentation," 2026, arXiv:2603.16044.

[18] S. Yang, *et al.*, "Vision-language-action instruction tuning: From understanding to manipulation," in *Proc. ICLR*, 2026.

[19] S. Lian, *et al.*, "LangForce: Bayesian decomposition of vision language action models via latent action queries," in *Proc. ICML*, 2026.

[20] Z. Zhan, *et al.*, "Stable language guidance for vision–language–action models," in *Proc. ACL*, 2026, pp. 4137–4159.

[21] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. Princeton University Press, 2021.

[22] X. Zhou, *et al.*, "LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization," 2025, arXiv:2510.03827.

[23] S. Fei, *et al.*, "LIBERO-Plus: A progressive robustness benchmark for visual-language-action models," in *Proc. IEEE/CVF CVPR*, 2026, pp. 38 574–38 583.

[24] D. Emukpere, R. Deffayet, and J.-M. Renders, "Robust skills, brittle grounding: Diagnosing restricted generalization in vision-language action policies via multi-object picking," 2026, arXiv:2602.24143.

[25] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Generalization through memorization: Nearest neighbor language models," in *Proc. ICLR*, 2020.

[26] J. Pari, N. Shafiullah, S. Arunachalam, and L. Pinto, "The surprising effectiveness of representation learning for visual imitation," in *Proc. RSS*, 2022.

[27] R. Li, *et al.*, "MAP-VLA: Memory-augmented prompting for vision-language-action model in robotic manipulation," in *Proc. IEEE ICRA*, 2026.

[28] J. Park, J. Park, T. Kim, S. Choi, D. Han, and S. Yun, "Retrieve, don't retrain: Extending vision language action models to new tasks at test time," 2026, arXiv:2606.15631.

[29] S. Jang, M. Jeon, M. Kim, S. Choi, D. Kim, and H. Yu, "RA-VLA: Retrieval-augmented VLA for test-time adaptation," in *Proc. ICML*, 2026.

[30] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang, "VIP: Towards universal visual reward and representation via value-implicit pre-training," in *Proc. ICLR*, 2023.

[31] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Proc. CoRL*, vol. 205, 2023, pp. 892–909.

[32] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman, "LIV: Language-image representations and rewards for robotic control," in *Proc. ICML*, vol. 202, 2023, pp. 23 301–23 320.

[33] V. Myers, B. Zheng, A. Dragan, K. Fang, and S. Levine, "Temporal representation alignment: Successor features enable emergent compositionality in robot instruction following," in *Adv. Neural Inf. Process. Syst.*, vol. 38, 2025, pp. 149 934–149 961.

[34] A. Radford, *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, vol. 139, 2021, pp. 8748–8763.

[35] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, arXiv:1807.03748.

[36] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 3942–3951.

[37] W. Li, R. Zhang, R. Shao, J. He, and L. Nie, "CogVLA: Cognition-aligned vision-language-action models via instruction-driven routing & sparsification," in *Adv. Neural Inf. Process. Syst.*, vol. 38, 2025, pp. 137 646–137 675.

[38] W. Song, *et al.*, "ReconVLA: Reconstructive vision-language-action model as effective robot perceiver," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 22, 2026, pp. 18 549–18 557.

[39] S. Liu, *et al.*, "RDT-1B: A diffusion foundation model for bimanual manipulation," in *Proc. ICLR*, 2025.

[40] Y. Wang, *et al.*, "VLA-Adapter: An effective paradigm for tiny-scale vision-language-action model," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 22, 2026, pp. 18 638–18 646.

[41] A. Vaswani, *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[42] L. Zhang, A. Rao, and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in *Proc. IEEE/CVF ICCV*, 2023, pp. 3836–3847.

[43] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. ICLR*, 2023.

[44] C. Chi, *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Robot. Res.*, vol. 44, no. 10–11, pp. 1684–1704, 2025.

[45] R. Knight, *et al.*, "Standard open SO-100 & SO-101 arms," Open-source hardware repository, 2024.

[46] B. Liu, *et al.*, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 44 776–44 791.